Ejercicio 1

Muestra en la página la fecha actual con formato DD/MM/AAAA y la hora HH:MM:SS. Obtén por separado día, mes, año, horas, minutos y segundos.

Ejercicio 2

Solicita al usuario una fecha en formato AAAA-MM-DD y muestra la fecha en un formato legible.

Ejercicio 3

A partir de una fecha introducida por el usuario, indica qué día de la semana es. Muestra el nombre del día en castellano.

Ejercicio 4

Solicita una fecha de nacimiento y calcula la edad en años cumplidos. Ten en cuenta si la persona ya ha celebrado su cumpleaños este año.

Ejercicio 5

Solicita una fecha futura y calcula cuántos días faltan desde hoy. Evita que las horas y minutos actuales alteren el cálculo de días naturales.

Ejercicio 6

Crea una función que reciba una fecha y una cantidad de días (positiva o negativa) y devuelva la fecha resultante. Comprueba cambios de mes, año y años bisiestos.

Ejercicio 7

Solicita dos fechas y calcula el número de días que hay entre ellas. Valida ambas fechas y contempla que la segunda pueda ser anterior a la primera.

Ejercicio 8

Crea un array de eventos con título y fecha. Muestra los eventos ordenados cronológicamente y diferencia los que ya han pasado de los que están pendientes.

Ejercicio 9

Solicita el color favorito y el idioma del usuario. Guarda ambos valores en localStorage y recupéralos al iniciar la aplicación. Añade una opción para eliminar las preferencias.

Ejercicio 10

Crea un array de números introducidos por el usuario. Guarda el array completo en una única clave de localStorage. Al recuperar los datos, muestra los números, su suma y su media.

Ejercicio 11

Partiendo del ejercicio anterior, permite añadir números y eliminar un número seleccionado. Cada modificación debe actualizar el array almacenado.

Ejercicio 12

Crea un objeto tipo JS usuario con nombre, edad y localidad. Guárdalo como JSON en localStorage y recupéralo para mostrar sus propiedades en HTML.

Ejercicio 13

Recupera el objeto usuario del ejercicio anterior y permite modificar su edad y localidad. Guarda de nuevo el objeto completo, sin perder las propiedades que no se hayan modificado.

Ejercicio 14

Crea un array de contactos (del tipo JS Objeto Notation). Cada contacto tendrá id, nombre, teléfono y correo. Guarda el array completo en localStorage y permite añadir, listar y eliminar contactos.

Ejercicio 15

Amplía la agenda: permite buscar contactos por nombre y ordenar el array alfabéticamente. Las operaciones deben realizarse sobre los datos recuperados y persistir los cambios cuando corresponda.

Ejercicio 16

Gestiona productos (del tipo JS Objeto Notation) con id, marca, modelo, kilómetros y fecha de adquisición. Guarda el array completo. Permite añadir productos y ordenar por marca o por kilómetros.

Ejercicio 17

Define una clase Persona con nombre, edad y un método mostrarInformacion(). Crea varias instancias y muestra sus datos en HTML.

Ejercicio 18

Guarda una instancia de Persona en localStorage. Recupera el contenido y muestra sus propiedades. Comprueba mediante instanceof si el objeto recuperado sigue siendo una instancia de Persona.

Ejercicio 19

Crea un método Persona.fromJSON(datos) que reciba los datos recuperados y devuelva una nueva instancia de Persona. Comprueba que el método mostrarInformacion() funciona después de reconstruirla.

Ejercicio 20

Crea un array de varias personas, conviértelo a JSON y guárdalo en localStorage. Al recuperarlo, reconstruye cada elemento como una instancia de Persona y muestra todas las personas.

Ejercicio 21

Define la clase Coche con marca, modelo y kilómetros. Añade métodos para mostrar información y actualizar kilómetros. Guarda un array de coches y reconstruye las instancias al recuperar los datos.

Ejercicio 22

Crea las clases Categoria y Producto. Cada producto pertenece a una categoría. Guarda un array de productos en localStorage y reconstruye tanto los productos como las categorías, manteniendo la relación entre ellos.

Ejercicio 23

Toma los ejercicios de fechas y crea un módulo fechas.js que exporte las funciones de validación y cálculo de fechas. En app.js importa las funciones y gestiona la interacción con el usuario y el DOM.

Ejercicio 24

Toma el ejercicio de productos y crea storage.js con funciones reutilizables para guardar, recuperar y eliminar datos de localStorage.

Ejercicio 25

Toma el ejercicio 20 y separa la solución en los módulos correspondientes para localStorage, app para coordinación e interacción con el usuario y funciones, donde filtraras las personas mayores de edad, las mayores que una edad dada por el usuario, las ordenaras alfabéticamente y por edad.